

React JS

Next.js

Material

MOBILE_005

NEXT.js

Introdução ao Next.js

Next.js



Agenda

- Criar um Projeto em Next.js
- Fazer o Deploy
- Hospedar gratuitamente na Vercel



Next.js

- Next.js é uma estrutura de JavaScript para desenvolvimento de aplicações web baseada em React.
- Foi criada pela Vercel e está disponível como um projeto de código aberto.



Recursos

1. Renderização do lado do servidor (SSR)
2. Geração de site estático (SSG)
3. Roteamento baseado em sistema de arquivos:
4. Suporte para TypeScript
5. API Routes
6. Hot Code Reloading



Renderização do lado do servidor (SSR)

- Isso permite que as aplicações sejam renderizadas no servidor antes de serem enviadas ao navegador, o que pode melhorar o desempenho e a otimização para motores de busca (SEO).



Geração de site estático (SSG):

- ▶ O Next.js permite que você gere sites estáticos a partir de seus aplicativos React. Isso é particularmente útil para sites com conteúdo que não muda frequentemente, já que permite que a página seja gerada uma vez e servida a cada pedido, resultando em tempos de carregamento de página mais rápidos.



Roteamento baseado em sistema de arquivos

- O Next.js segue a convenção sobre a configuração para as rotas, onde os arquivos na pasta "pages" se tornam rotas automaticamente.



Suporte para TypeScript



- O Next.js tem suporte integrado para TypeScript, uma variante tipada do JavaScript.



API Routes

- ▶ Permite a criação de rotas de API REST ou API endpoints dentro do mesmo projeto Next.js, facilitando a construção de um back-end junto com o seu front-end



Hot Code Reloading

- Recarrega automaticamente a página sempre que você faz uma alteração no código.

Empresas (usam Next.js)

- Netflix,
- Github,
- Starbucks,
- Docker,
- Twitch,
- Uber,
- Airbnb,
- TikTok,
- Nike, etc...

Deploy

Next.js





Deploy

- "Deploy" é um termo amplamente usado na área de desenvolvimento de software para se referir ao processo de fazer um aplicativo ou um serviço estar disponível para uso.
- Em termos mais práticos, o deploy pode ser visto como a série de etapas que são executadas para mover o código do software de um ambiente de desenvolvimento ou teste para um ambiente de produção, onde o software pode ser acessado e utilizado pelos usuários finais.



DEPLOY DE SOFTWARE

Do código à produção: as etapas para entregar valor com segurança e confiabilidade.



1

COMPILAÇÃO

Em alguns casos, o código fonte do aplicativo é compilado para criar um executável ou outros artefatos que podem ser executados em um servidor ou outra infraestrutura.



2

EMPACOTAMENTO

Os artefatos de software criados são frequentemente agrupados em um pacote que pode ser facilmente movido entre ambientes.



3

TRANSPORTE

O pacote de software é então movido do ambiente de desenvolvimento ou teste para o ambiente de produção.



4

INSTALAÇÃO

O pacote de software é instalado no ambiente de produção, o que pode envolver a extração dos artefatos do pacote e a realização de outras tarefas de configuração.



5

CONFIGURAÇÃO

Quaisquer configurações necessárias para o ambiente de produção são aplicadas ao software. Isso pode incluir a configuração de conexões de banco de dados, chaves de API, variáveis de ambiente, etc.



6

ATIVAÇÃO

O software é então iniciado e começa a aceitar solicitações de usuários ou outros sistemas.



OBJETIVO DO DEPLOY:

Garantir que o software seja entregue ao ambiente de produção de forma padronizada, segura, repetível e com o mínimo de risco, proporcionando valor contínuo ao negócio e aos usuários.



Laboratório

- Desenvolvimento de uma aplicação SPA usando o React JS. Nome da aplicação sample06



Primeiro passo:

Criar um repositório no github, chamado sample06

Owner * Repository name *

 maromo71 ▾ / sample06

✔ sample06 is available.

Great repository names are short and memorable. Need inspiration? How about [animated-doodle?](#)

Description (optional)

Passo 1: Configuração do ambiente de desenvolvimento

```
6 pasta(s) 160.417.779.712 bytes disponíveis
```

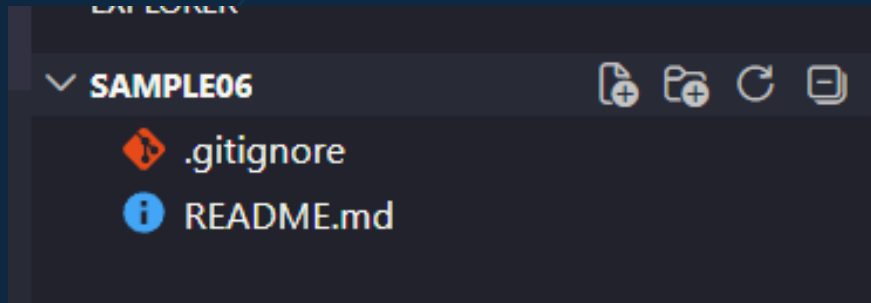
```
C:\javascript\react01>git clone https://github.com/maromo71/sample06|
```

```
receiving objects: 100% (4/4), done.
```

```
C:\javascript\react01>cd sample06
```

```
C:\javascript\react01\sample06>code .
```

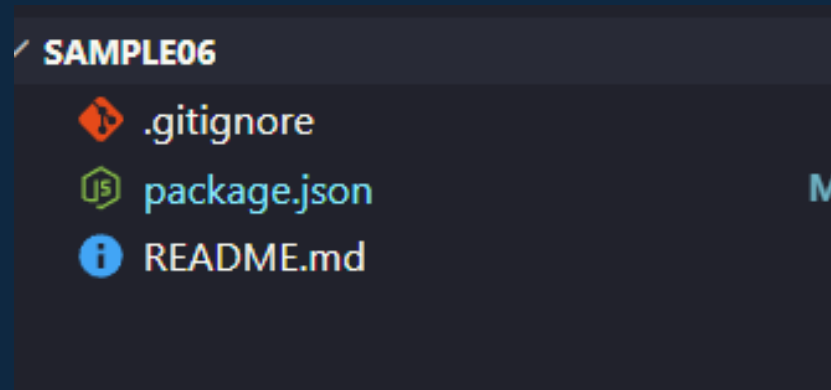
Passo 2: Estrutura do projeto clonado



- Neste momento precisamos ter instalado na máquina:
 - Node.js
 - Next.js
 - React.js

Passo 3

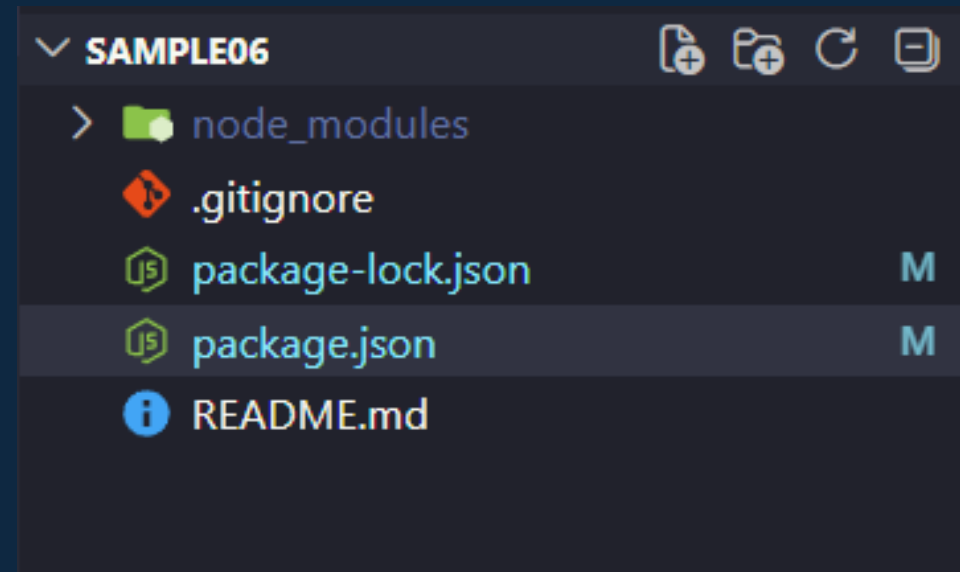
- No diretório da aplicação, no terminal do VS CODE, digite:
`npm init -y`



Passo 4

- Para instalar a dependência do Next.js, digite o comando:

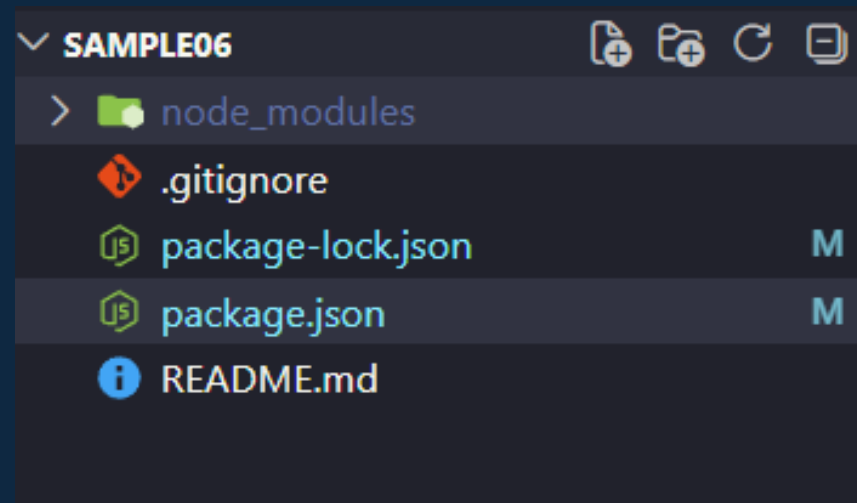
npm install next



Passo 5

- Para instalar a dependência do React, digite o comando:

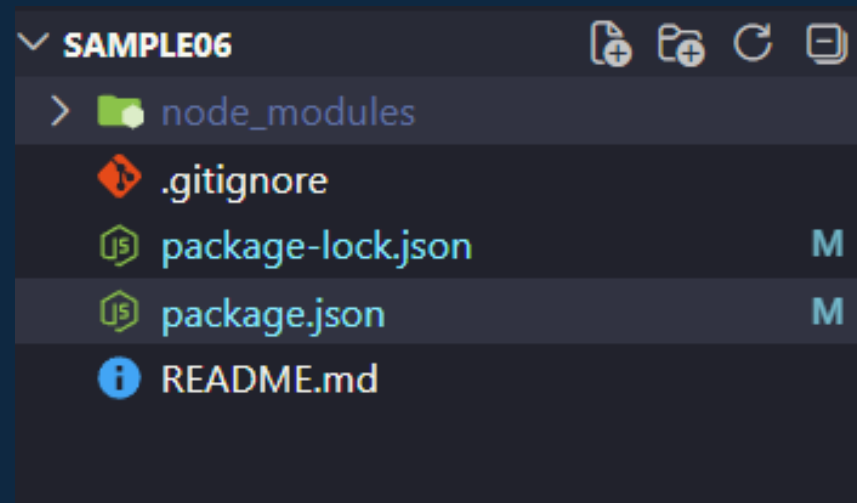
npm install react



Passo 6

- Para instalar a dependência do React-dom, digite o comando:

npm install react-dom



Passo 7 - Criando nossa página estática

- No package.json:
 - Alterando a linha de script, incluindo a dev, que simplesmente vai rodar o comando "next dev"
 - Altere a linha type de "common.js" para "module"

```
✓ Debug  
"scripts": {  
  "dev": "next dev",  
  "build": "next build",  
  "start": "next start"  
},
```

```
15   "keywords": [],  
16   "author": "",  
17   "license": "ISC",  
18 |  "type": "module",
```

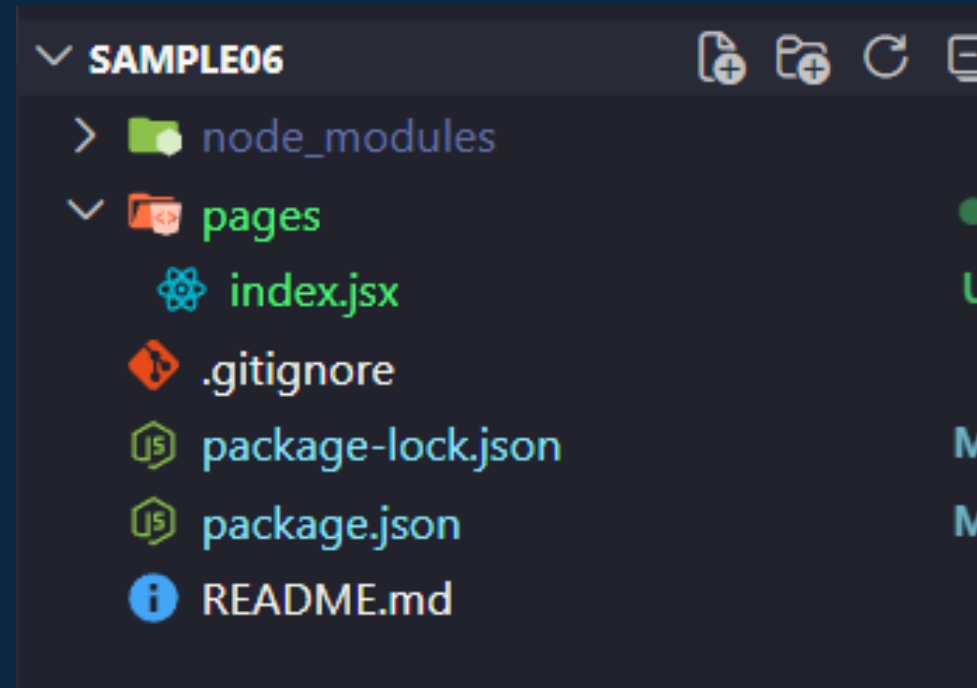


Rotas no Next.js

- Você não precisa configurar as rotas do seu projeto, porque para cada arquivo dentro da pasta pages (que criaremos) virá automaticamente uma rota e uma página.

Passo 8 - Criando a pasta Pages

- Criando a pasta pages e dentro dela o arquivo index.jsx.



Componente Home

- Dentro do arquivo index.js, vamos criar nosso componente React que representa a página home da nossa aplicação.
- Como dentro de um arquivo Javascript podemos ter vários componentes React, várias funções tradicionais, **então como o Next.js vai saber qual componente/função representa a nossa página ?**
 - Simples. Basta usar o novo padrão de módulo do Javascript e exportar por default o componente para que seja renderizado como página.

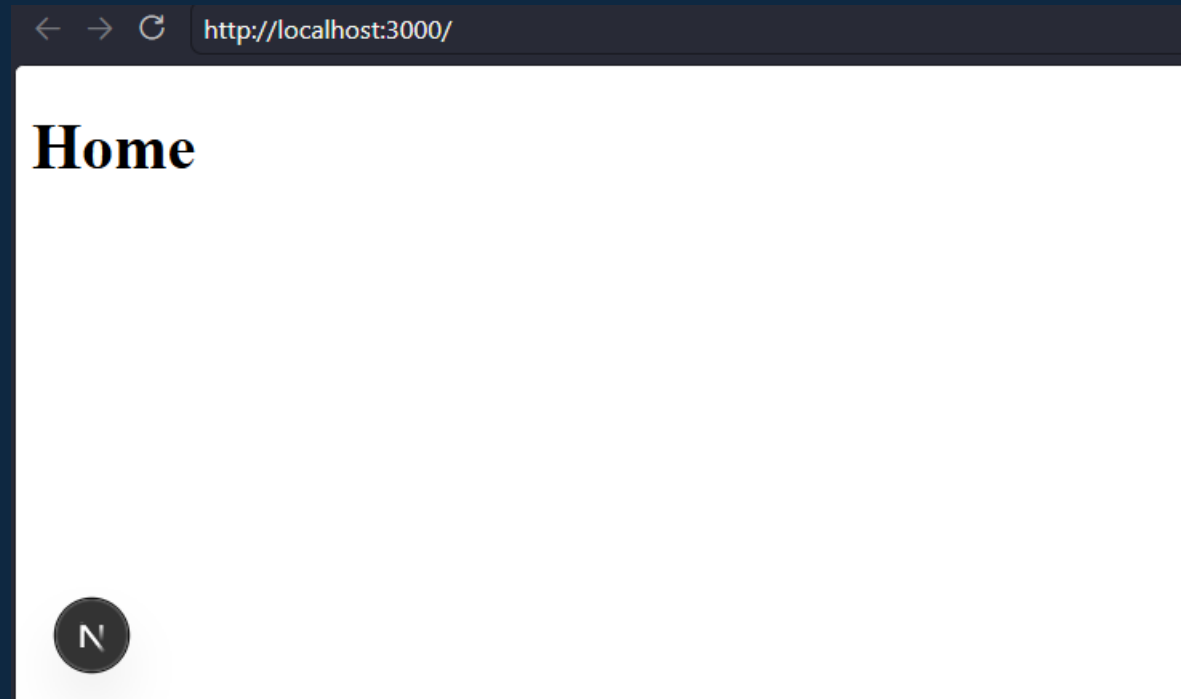
Passo 9 - Componente: Home



```
export default function Home(){  
  return (  
    <div>  
      <h1>  
        Home  
      </h1>  
    </div>  
  )  
}
```

Rodando Localmente

➡ `npm run dev`



Home

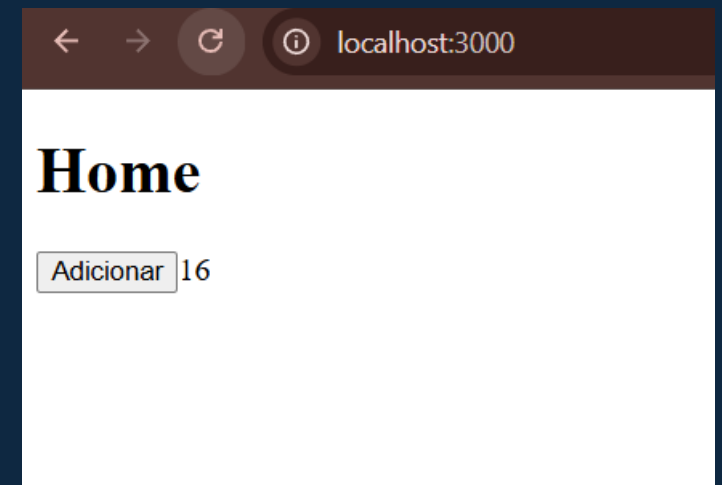
- Vamos alterar o nosso componente Home para que ele funcione como um contador, que ao clicar em um botão chamado Adicionar, execute alteração no estado da aplicação.
- Neste exemplo, sempre que se clicar no botão adicionar ele vai manter o estado do componente, de maneira que ao executar a função **adicionarContador** ele incrementa um na variável contador.
- Você pode fazer ajustes no código e verificar as alterações sem perder o estado anterior.
- Experimente acrescentar uma tag H3 para alterar o título da página e salve. Ele alterará a div que contém a palavra Home e mantém o valor do contador.

Home

```
import { useState } from 'react'
export default function Home() {
  function Contador() {
    const [contador, setContador] = useState(1)

    function adicionar() {
      setContador(contador + 1)
    }

    return (
      <div>
        <button onClick={adicionar}>Adicionar</button>
        <span>{contador}</span>
      </div>
    )
  }
  return (
    <div>
      <h1>Home</h1>
      <Contador />
    </div>
  )
}
```





Pronto

- Com essa página estática criada, que tal colocarmos ela no ambiente de produção. Apesar de simples, podemos armazená-la na nuvem em um endereço de verdade na net que você poderá acessar de qualquer lugar.
- A primeira coisa a se fazer é parar o servidor de desenvolvimento. (CTRL + S)
- Agora vamos colocar as alterações na branch principal do github.

Vamos colocar na nuvem GITHUB



Colocando na branch principal



Sequencia de comandos no terminal:

```
git add -A
```

```
git commit -m "Primeiro site na nuvem"
```

```
git push origin main
```

Repositório no Github



maromo71 / sample06 Public

<> Code Issues Pull requests Actions Projects Security Insights

main 1 branch 0 tags

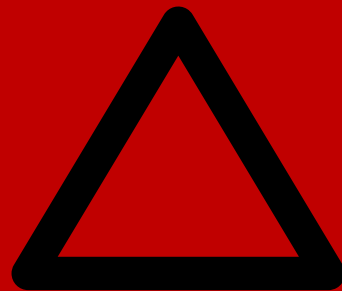
maromo71 Primeiro site na nuvem

pages	Primeiro site na nuvem
.gitignore	Initial commit
README.md	Initial commit
package-lock.json	Primeiro site na nuvem
package.json	Primeiro site na nuvem



VERCEL
hospedando

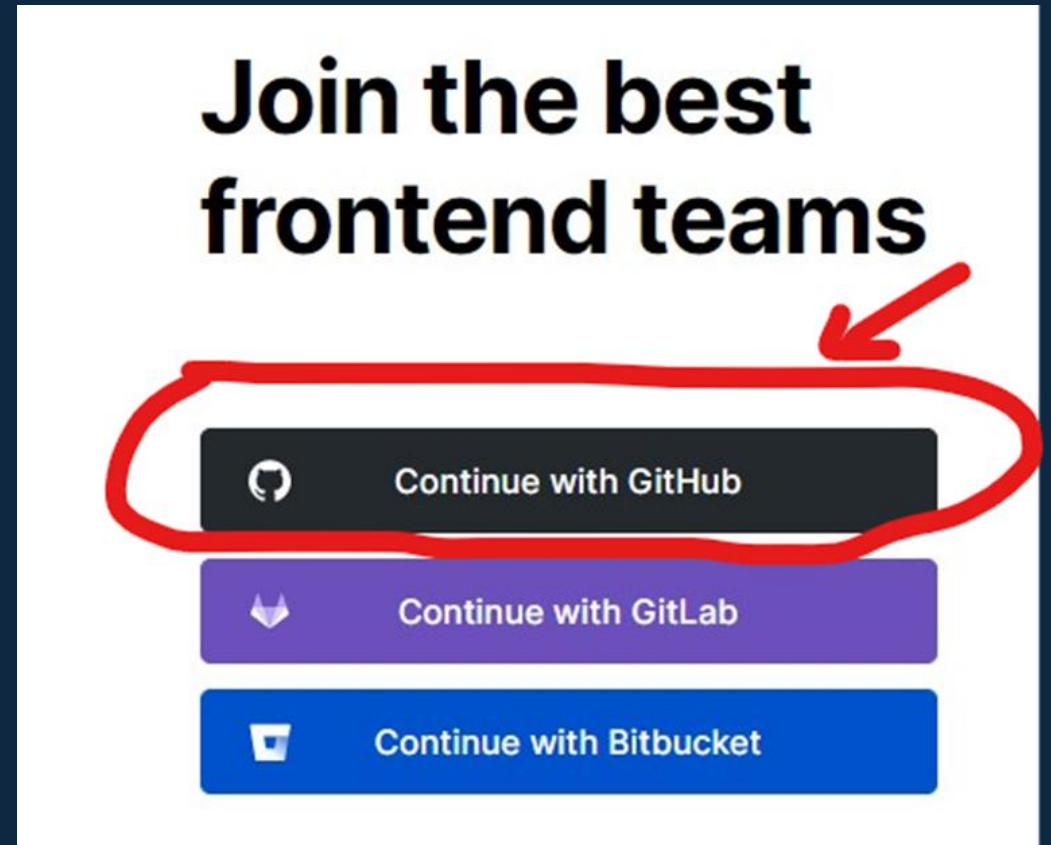
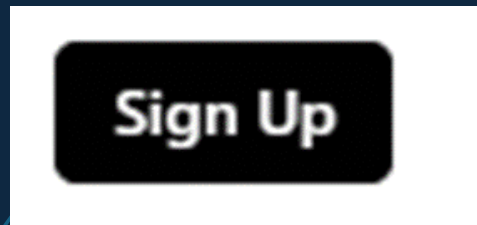
Develop.
Preview.
Ship.

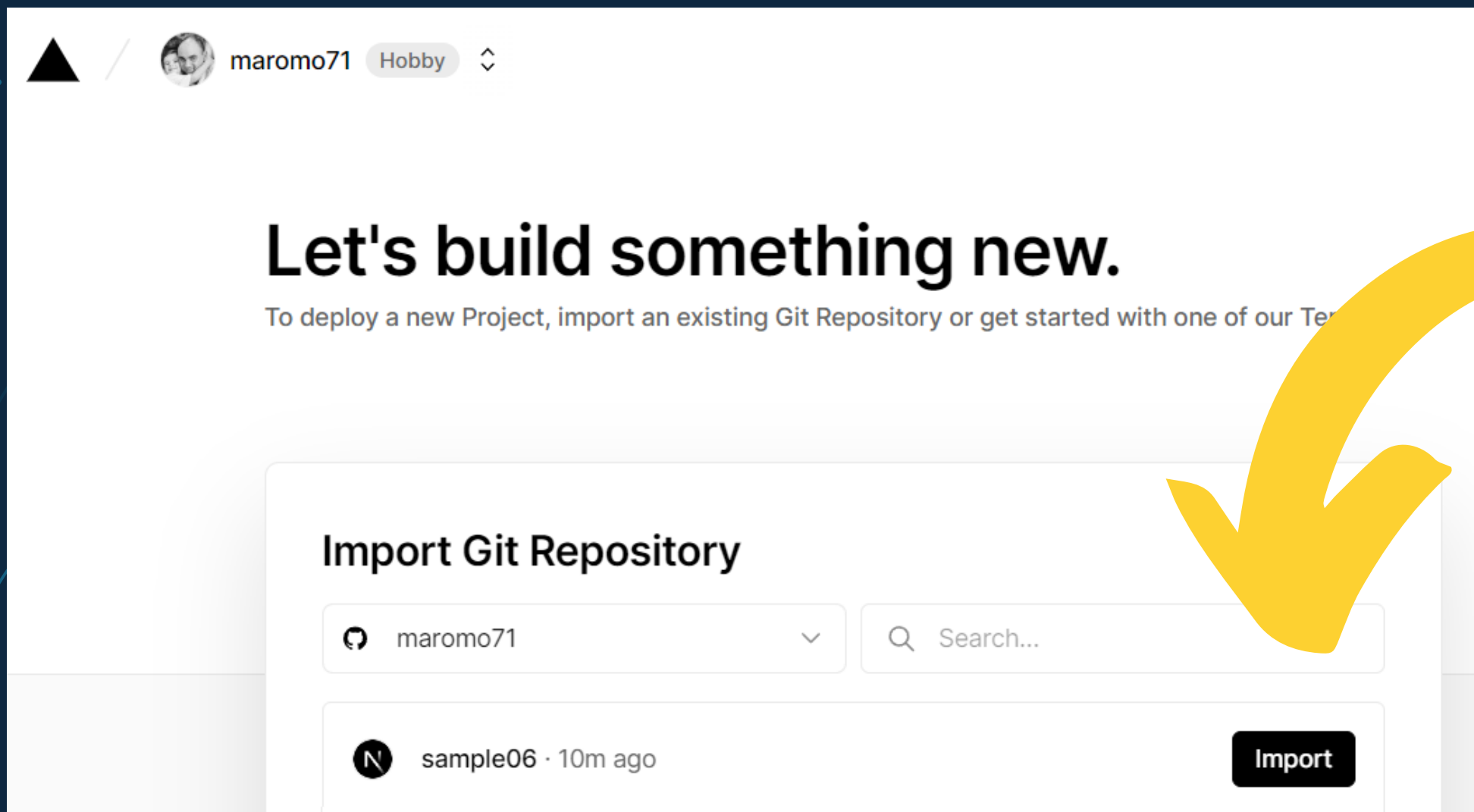


Acesse o site

<https://vercel.com/>

E entre no Sing Up para se registrar:





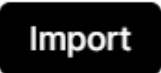
▲ /  maromo71 Hobby ⌵

Let's build something new.

To deploy a new Project, import an existing Git Repository or get started with one of our Templates

Import Git Repository

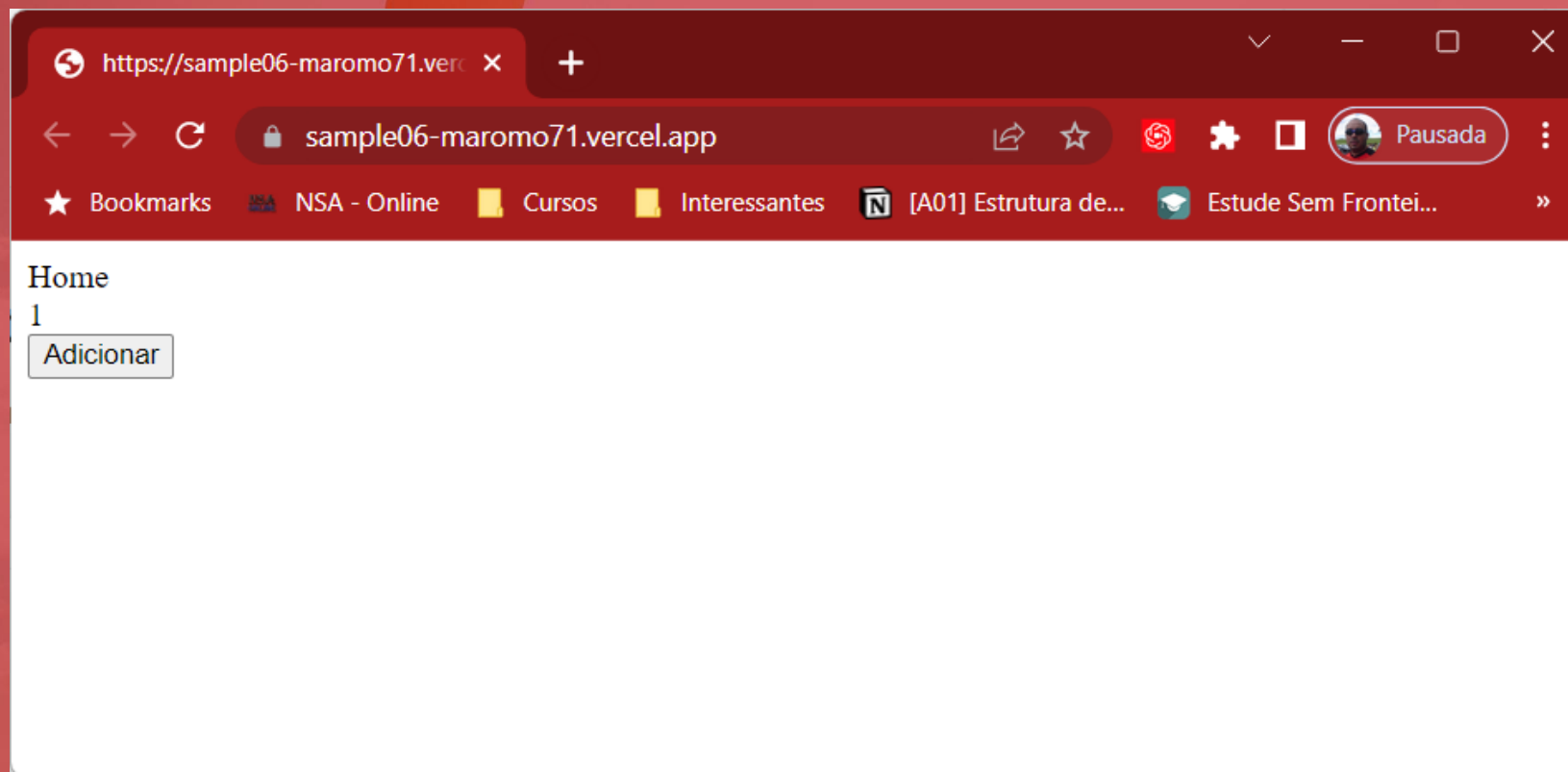
 maromo71 ▼  Search...

 sample06 · 10m ago 

**Após este processo de importação, escolha
Deploy**

Pronto

<https://sample06-maromo71.vercel.app/>



Resultado da Execução



Conclusão

- O Next.js se consolida como um dos frameworks mais poderosos para React, unindo simplicidade de configuração com alta performance. Vimos que recursos nativos como SSR, SSG e o roteamento baseado em arquivos resolvem problemas complexos de infraestrutura e SEO de forma automática.
- Além disso, a integração direta com a plataforma da Vercel simplifica o processo de deploy, permitindo que sua aplicação saia do ambiente local para a nuvem em questão de minutos.



Agora
você será
capaz de:

- Compreender os conceitos fundamentais e as principais vantagens de usar o Next.js.
- Configurar o ambiente e inicializar um projeto Next.js do zero utilizando o npm.
- Criar rotas e páginas automaticamente utilizando a estrutura da pasta "pages".
- Desenvolver componentes funcionais em React manipulando estados (como o contador).
- Versionar seu código e enviá-lo para um repositório no GitHub.
- Realizar o deploy da sua aplicação na nuvem de forma gratuita utilizando a Vercel.



Introdução a props
Propriedades

Next.js React



Passando Dados com Props

Comunicação entre Componentes

Agenda

-  O que são "Props"? Entendendo as propriedades.
-  A Importância das Props: Por que usá-las?
-  Como Usar Props: O fluxo Pai-Filho.
-  Laboratório: Criando um Baralho de Signos.

O que são "Props"?

"**Props**" (abreviação de propriedades) são a forma de passar dados de um componente **PAI** para um componente **FILHO** em React.

Pense nelas como os "argumentos" de uma função.

Elas são **SOMENTE LEITURA**. Um componente filho não pode alterar as props que recebe.

React Props

Passando dados entre componentes

Props (propriedades) são uma forma de passar dados de um componente pai para um componente filho em React. Elas tornam os componentes reutilizáveis e dinâmicos.



Props são como **argumentos** de funções, mas usados entre componentes.

NEXT.JS

Mesma lógica do React, funcionando perfeitamente no Next.js!

1 Componente Pai

O componente pai envia os dados via props.

```
app/page.jsx

import Card from '@components/Card'

export default function Home() {
  return (
    <div>
      <h1>Bem-vindo ao Next.js!</h1>
      <Card
        nome="Marcos"
        curso="Desenvolvimento Web"
        imagem="/perfil.jpg"
      />
    </div>
  )
}
```

O pai passa os dados para o filho



2 Componente Filho

O componente filho recebe os dados via props.

```
components/Card.jsx

export default function Card(props) {
  return (
    <div className="card">
      <img
        src={props.imagem}
        alt={props.nome}
        className="foto"
      />
      <h2>{props.nome}</h2>
      <p>{props.curso}</p>
    </div>
  )
}
```



Resultado na tela

O componente filho exibe os dados recebidos.



Marcos

Desenvolvimento Web



Dicas importantes

- ✓ Props são somente leitura (não devem ser alteradas pelo filho).
- ✓ Podem ser strings, números, objetos, arrays, funções, etc.
- ✓ Tornam os componentes mais reutilizáveis.
- ✓ No Next.js, funcionam da mesma forma que no React.
- ✓ Use nomes descritivos para facilitar a manutenção.



Exemplos de tipos de props

Você pode passar diferentes tipos de dados.

```
// String
<Mensagem texto="Olá, Next.js!" />

// Número
<Contador valor={10} />

// Objeto
<Usuario dados={{ id: 1, nome: "Marcos" }} />

// Função
<Botao onClick={() => alert('Clique!')} />
```

A Importância das Props



Comunicação

Permitem a comunicação e a passagem de dados (estado, funções, etc.) do componente Pai para o Filho.



Reutilização

Tornam componentes reutilizáveis. Você pode usar o mesmo componente (ex: Card) várias vezes com dados diferentes.

Como Usar Props: O Fluxo

1. Componente Pai (ex: App.jsx)

// Passe props como "atributos"
HTML

```
function App() {  
  return (  
    <MeuComponente  
      nome="Maromo"  
      idade={30}  
    />  
  );  
}
```

2. Componente Filho (ex: MeuComponente.jsx)

// Receba as props como um objeto
function MeuComponente(props) {
 return (
 <p>

Olá, {props.nome}!
Você tem {props.idade} anos.
</p>

);
}

// Dica: Use desestruturação
// function MeuComponente({ nome, idade }) {
// return Olá, {nome}!;
// }



Laboratório

Baralho de
Signos

Repositório Vazio (github – clone com o nome signos)

No terminal:

Na raiz do disco (c:)

Digite:

```
git clone https://github.com/<<seu-usuário>>/signos
```

Em seguida:

Entre no diretório clonado: `cd signos`

Agora, acesso o VS CODE

```
C:\signos> code .
```

`next`

Repositório Vazio para aplicação Next

No terminal:

```
npm init -y
```

Em seguida:

```
npm install react
```

Depois:

```
npm install react-dom
```

Por fim:

```
npm install next
```

Modifiez le package.json

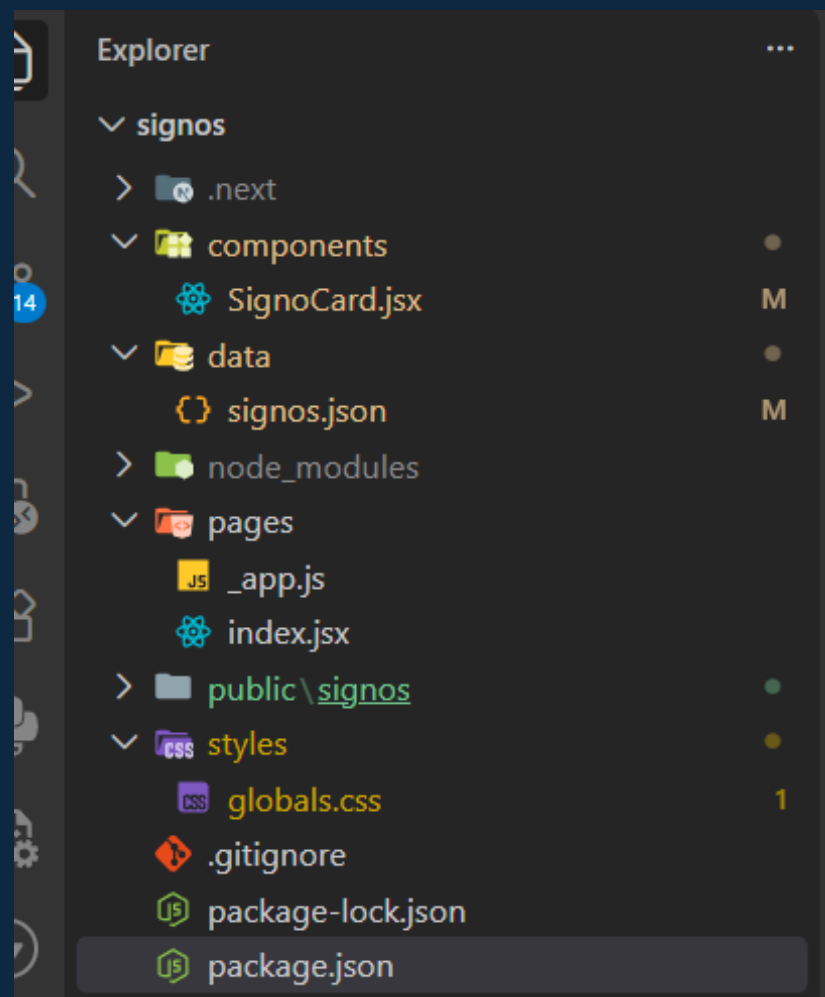
```
16 "keywords": [],  
17 "author": "",  
18 "license": "ISC",  
19 "type": "module",
```



```
  > Debug  
7  "scripts": {  
8    "dev": "next dev",  
9    "build": "next build",  
10   "start": "next start"  
11 },
```



Estrutura das pastas



Lab: Passos 1 – Componente `SignoCard`

src/components/SignoCard.jsx

```
import React from 'react';
import './style.css';

// Recebendo o objeto props
export default function SignoCard(props) {
  return (
    <div className="signo-card">
      /* Usando os dados das props */
      <h2>{props.signo}</h2>
      <img src={props.imagem}
alt={props.signo} />
      <p>
        {props.dataInicio} - {props.dataFim}
      </p>
    </div>
  );
}
```

Lab: Passo 3 – Usando no `App.jsx`

Reutilizando o Componente

1. Importamos o SignoCard.
2. Usamos o várias vezes.
3. Cada "instância" recebe um conjunto **diferente** de props, gerando um card único.

Observação: Para as imagens funcionarem, elas devem estar na pasta public do seu projeto (ex: public/aries.png).

```
import SignoCard from './components/SignoCard';

function App() {
  return (
    <div className="App">
      <h1>Baralho dos Signos</h1>

      <div className="signos-container">
        {/* Passando props para o 1º card */}
        <SignoCard
          signo="Áries"
          imagem="https://placeholder.co/100x100/f07178/white?text=Aries"
          dataInicio="21/03"
          dataFim="20/04"
        />

        {/* Passando props para o 2º card */}
        <SignoCard
          signo="Touro"
          imagem="https://placeholder.co/100x100/c3e88d/black?text=Touro"
          dataInicio="21/04"
          dataFim="21/05"
        />

        {/* Adicione mais signos... */}
      </div>
    </div>
  );
}

export default App;
```

Resultado

Você agora tem um componente SignoCard reutilizável!

Podemos criar 12 cards diferentes usando o **mesmo** componente, apenas mudando as **props**.

Melhorias

Você agora tem um componente SignoCard reutilizável!

Usar um arquivo json com dados e usar a função map para percorrer um vetor e executar a função para cada item. Veja a figura:



Entendendo o Código

Renderização de uma lista de signos em React

Este código usa o método `.map()` para percorrer um array de dados (`signosData`) e renderizar um componente `SignoCard` para cada signo do zodiaco.



Conceitos envolvidos

JSX

`map()`

Componentes

Props

Renderização de listas

"Pequenos componentes, grandes resultados!"

1

Container principal

A `<div>` com a classe `signos-container` é o elemento pai que agrupa todos os cards dos signos.

2

Método `.map()`

O `signosData.map()` percorre todos os itens do array `signosData` e executa uma função para cada item, retornando o componente `SignoCard`.

3

Parâmetro `item`

Em cada iteração, o `item` representa um objeto do array, contendo as informações de um signo (ex.: nome, imagem, datas).

4

Componente `SignoCard`

Para cada item, um componente `<SignoCard>` é renderizado, recebendo as informações através de props.

5

Props passadas

São enviadas quatro props para o componente: `signo`, `imagem`, `dataInicio` e `dataFim`.

O Código

JSX (React)



```
1 <div className="signos-container">
2   {signosData.map((3 item) => (
4     <SignoCard
5       signo={item.signo}
        imagem={item.imagem}
        dataInicio={item.dataInicio}
        dataFim={item.dataFim}
      />
    )
  )}
</div>
```



Resultado na tela



Áries

21/03 - 19/04



Touro

20/04 - 20/05



Gêmeos

21/05 - 20/06



Câncer

21/06 - 22/07



Leão

23/07 - 22/08



Virgem

23/08 - 22/09



Libra

23/09 - 22/10



Escorpião

23/10 - 21/11



Sagitário

22/11 - 21/12



Capricórnio

22/12 - 19/01



Aquário

20/01 - 18/02



Peixes

19/02 - 20/03



Estrutura de dados (exemplo)

O array `signosData` pode ser algo assim:

```
const signosData = [
  {
    signo: "Áries",
    imagem: "/imagens/aries.png",
    dataInicio: "21/03",
    dataFim: "19/04"
  },
  ...
];
```



Como funciona

- 1 O React lê o array `signosData`.
- 2 O método `.map()` percorre cada item do array.
- 3 Para cada item, o componente `SignoCard` é renderizado com as props correspondentes.
- 4 O resultado é uma lista de cards exibindo todos os signos.



Dica

Em listas, é recomendado adicionar a prop `key` no componente:

```
signosData.map((item) => (
  <SignoCard key={item.signo} {...} />
))
```



Boas práticas

- ✓ Use a prop `key` em listas.
- ✓ Mantenha os componentes pequenos e reutilizáveis.
- ✓ Organize os dados em um arquivo ou serviço (ex.: JSON, API).
- ✓ Use nomes de classes significativos (ex.: `signos-container`).

Código bem organizado gera aplicações incríveis!"



Perguntas?

Obrigado!

Fim



Prof. Maromo